

CMS - Complaint Management System

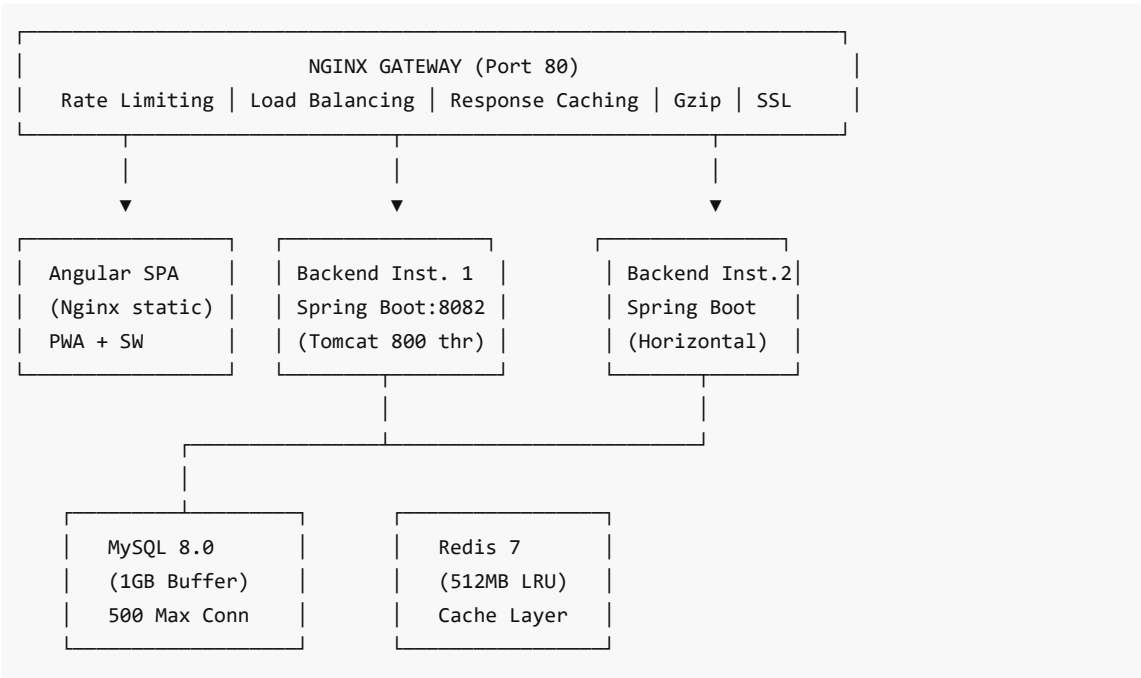
Architecture & Feature Documentation

Version: 1.0.0
Stack: Angular 21 + Spring Boot 3.2.5 + MySQL 8 + Redis 7
Target Scale: 1 Million users, pan-India customer-facing portal
Last Updated: 2026-05-14

Table of Contents

- 1. [System Architecture Overview](#)
- 2. [Technology Stack](#)
- 3. [Project Structure](#)
- 4. [Feature List](#)
- 5. [Frontend Architecture](#)
- 6. [Backend Architecture](#)
- 7. [Database Design](#)
- 8. [Scalability & Performance](#)
- 9. [Infrastructure & Deployment](#)
- 10. [API Reference](#)
- 11. [Developer Guide - How to Modify](#)

1. System Architecture Overview



Request Flow:

- 1. User hits `http://domain.in` → Nginx Gateway
- 2. Static assets → Served from Angular build (1yr cache)

- 3. API calls (/api/*) → Load balanced across backend instances
- 4. Backend checks Redis cache → If miss, queries MySQL
- 5. Response flows back through Nginx (may be cached at gateway level)

2. Technology Stack

Layer	Technology	Version	Purpose
Frontend	Angular	21.2.0	SPA Framework
UI Library	PrimeNG	21.1.6	UI Components + Icons
Backend	Spring Boot	3.2.5	REST API Framework
Language	Java	17	Backend Runtime
Database	MySQL	8.0	Primary Datastore
Cache	Redis	7	Distributed Cache
Gateway	Nginx	1.27	Reverse Proxy + LB
Container	Docker	Multi-stage	Deployment
Rate Limiting	Bucket4j	8.10.1	Per-IP Throttling
Monitoring	Actuator + Prometheus	-	Metrics & Health
Document Gen	jsPDF, docx, JSZip	-	PDF/Word Export
ORM	Hibernate/JPA	-	Database ORM
Build	Maven + Angular CLI	-	Build Tools

3. Project Structure

Frontend (cms-frontend/)

```
cms-frontend/
├─ public/
│   ├── manifest.json      # PWA manifest (icons, shortcuts)
│   ├── sw.js              # Service Worker (offline, bg sync)
│   ├── offline.html       # Offline fallback page
│   ├── rbi-seal.svg       # Brand assets
│   └── rbi-logo.svg
├─ src/
│   ├── index.html         # PWA meta tags, SW registration
│   ├── environments/
│   │   ├── environment.ts  # Dev: apiUrl → http://localhost:8082/api
│   │   └── environment.prod.ts # Prod: apiUrl → /api (relative)
│   └── app/
│       ├── app.routes.ts   # Lazy-loaded routes
│       └── app.config.ts    # Providers (HttpClient, Router)
```

```

|   |   | components/
|   |   | | layout/           # Shell (Header + Footer + router-outlet)
|   |   | | header/          # Top strip + Main nav + Hamburger menu
|   |   | | footer/          # Site footer
|   |   | | home/            # Landing page + Dashboard (logged-in)
|   |   | | file-complaint/   # Multi-step complaint form
|   |   | | track-complaint/  # Track by complaint number
|   |   | | physical-complaint/ # OCR-based physical filing
|   |   | | email-simulation/  # Email-to-complaint demo
|   |   | | └ dynamic-field/   # Dynamic form field renderer
|   |   | └ services/
|   |   | | cms.service.ts      # HTTP API client
|   |   | | complaint-parser.service.ts # NLP text → fields
|   |   | | complaint-text-processor.ts # Hindi→English + parsing
|   |   | | complaint-store.service.ts # LocalStorage state
|   |   | | └ ocr.service.ts    # Mock OCR extraction
|   |   | └ models/
|   |   | | form-schema.model.ts # Form config interfaces
| Dockerfile                # Multi-stage: node build → nginx serve
| nginx.conf                 # Static serving config
| angular.json               # Build config, budgets
└ package.json

```

Backend (cms-backend/)

```

cms-backend/
└ src/main/java/com/hrms/cms/
  |   |   | CmsApplication.java      # Spring Boot entry point
  |   |   | └ config/
  |   |   | | AsyncConfig.java       # Thread pool (10-50 threads)
  |   |   | | CorsConfig.java        # CORS (env-aware origins)
  |   |   | | DataInitializer.java    # Seed categories, banks, form
  |   |   | | GlobalExceptionHandler.java # @RestControllerAdvice
  |   |   | | RateLimitFilter.java    # Per-IP Bucket4j (100 req/s)
  |   |   | | └ RedisCacheConfig.java  # Per-cache TTL config
  |   |   | └ controller/
  |   |   | | ComplaintController.java # CRUD + paged + track
  |   |   | | BankController.java      # List banks by type
  |   |   | | CategoryController.java  # Hierarchical categories
  |   |   | | DashboardController.java # Aggregated stats
  |   |   | | EmailSimulationController.java # Email demo endpoints
  |   |   | | └ FormConfigController.java # Dynamic form schema
  |   |   | └ service/
  |   |   | | ComplaintService.java    # Core business logic
  |   |   | | EmailSimulationService.java # Email thread simulation
  |   |   | | └ FormConfigService.java  # Form schema management
  |   |   | └ dto/
  |   |   | | FileComplaintRequest.java
  |   |   | | UpdateComplaintRequest.java
  |   |   | | DashboardResponse.java
  |   |   | | └ EmailReplyWithFormRequest.java

```

```

| | | └─ IncomingEmailRequest.java
| | └─ entity/
| | | └─ Complaint.java           # Main entity (8 indexes)
| | | └─ ComplaintTimeline.java   # Audit trail per complaint
| | | └─ ComplaintAttachment.java # File attachments
| | | └─ ComplaintCategory.java   # Hierarchical categories
| | | └─ Bank.java                # Regulated entities
| | | └─ FormConfig.java          # JSON form schemas
| | | └─ SimulatedEmail.java      # Email threads
| | └─ repository/
| | | └─ ComplaintRepository.java
| | | └─ ComplaintTimelineRepository.java
| | | └─ ComplaintAttachmentRepository.java
| | | └─ ComplaintCategoryRepository.java
| | | └─ BankRepository.java
| | | └─ FormConfigRepository.java
| | | └─ SimulatedEmailRepository.java
| └─ src/main/resources/
| | └─ application.yml           # 3 profiles: default, dev, prod
| └─ Dockerfile                  # Multi-stage JDK→JRE
| └─ pom.xml

```

Infrastructure (cms-infra/)

```

cms-infra/
└─ docker-compose.yml      # Full stack: MySQL + Redis + 2x Backend + Frontend + Nginx
└─ nginx/nginx.conf        # API Gateway config
└─ mysql/init.sql          # DB creation + InnoDB tuning
└─ .env.example            # Environment variables template

```

4. Feature List

4.1 Customer-Facing Features

#	Feature	Component	Description
1	File a Complaint	file-complaint	Multi-step wizard form (3 steps) with dynamic fields driven by backend JSON schema
2	Track Complaint	track-complaint	Search by complaint number, view status + timeline
3	Physical Filing (OCR)	physical-complaint	Upload physical complaint letter → OCR extracts text → NLP parses fields → auto-fills form
4	Hindi Language Support	complaint-text-processor	Detects Hindi text, translates to English, extracts structured fields
5	Email-based Filing	email-simulation	Simulates filing complaint via email (incoming email → auto-generate complaint)
6	Pre-filing Validation	file-complaint	Modal asking if user already complained to bank (eligibility check)

7	Dashboard (Logged-in)	home	Complaint history table with status badges, sort, filter, and action buttons
8	Landing Page	home	Hero section, complaint types, What We Do, Stats, Press Releases, Education, FAQ
9	PWA Support	manifest.json + sw.js	Install to home screen, offline page, background sync for pending complaints
10	Mobile Responsive	All components	Fully responsive design with hamburger nav, bottom-sheet modals, touch-friendly

4.2 Backend/System Features

#	Feature	File	Description
11	Dynamic Form Config	FormConfigService	Forms are JSON-driven from DB, no code change needed to add/modify fields
12	Hierarchical Categories	ComplaintCategory	Parent-child category tree with sort order
13	Rate Limiting	RateLimitFilter	Per-IP: 100 req/s + 3000/min burst, returns 429 JSON
14	Redis Caching	RedisCacheConfig	TTL per cache: dashboard(2m), categories(1h), form-config(6h), email-stats(3m)
15	Async Timeline Writes	AsyncConfig + ComplaintService	Timeline audit entries written asynchronously (non-blocking to user)
16	Paginated APIs	ComplaintController	/complaints/paged with page, size, status filter, search
17	Database Indexing	All entities	8 indexes on Complaint, composite index on status+createdAt
18	Connection Pooling	application.yml	HikariCP: 50 (dev) / 100 (prod) connections with leak detection
19	Horizontal Scaling	docker-compose.yml	2 backend instances, Nginx load balancing (least_conn)
20	Monitoring	Actuator + Prometheus	Health checks, metrics, Prometheus scrape endpoint
21	Global Exception Handler	GlobalExceptionHandler	Consistent JSON error responses for all exceptions
22	Data Seeding	DataInitializer	Auto-seeds 10 categories, 12 banks, and complaint form schema on first run
23	Gzip Compression	Nginx + Tomcat	Response compression for JSON, HTML, CSS, JS

24	Security Headers	Nginx	X-Frame-Options, CSP, X-XSS-Protection, Referrer-Policy
----	------------------	-------	---

5. Frontend Architecture

5.1 Routing (Lazy Loaded)

/ (Layout shell)		
— /	→ HomeComponent	(Landing or Dashboard)
— /file-complaint	→ FileComplaintComponent	
— /track-complaint	→ TrackComplaintComponent	
— /email-simulation	→ EmailSimulationComponent	
— /physical-complaint	→ PhysicalComplaintComponent	

All routes are lazy-loaded via `loadComponent()` for optimal bundle splitting.

5.2 Component Responsibilities

Component	Key Logic
LayoutComponent	Shell with <app-header> + <router-outlet> + <app-footer>
HeaderComponent	Top utility strip (theme, accessibility, language, user info) + Main navigation with mobile hamburger
HomeComponent	Two views: (a) Landing page for anonymous users (b) Dashboard with complaint table for logged-in users
FileComplaintComponent	Fetches form schema from API → renders multi-step wizard using <code>DynamicFieldComponent</code> → submits complaint
DynamicFieldComponent	Renders a single form field based on <code>FormField</code> model (text, select, radio, file, etc.)
TrackComplaintComponent	Input complaint number → calls track API → displays status + timeline
PhysicalComplaintComponent	File upload → OCR → Text processing (Hindi detection + translation) → NLP parsing → Auto-fill form
EmailSimulationComponent	Simulates incoming email → backend creates complaint → shows thread view

5.3 Service Layer

Service	Purpose
CmsService	Central HTTP client for all backend API calls
ComplaintParserService	Dictionary-based NLP: extracts entity type, bank, category, name, phone, email, pincode, account, amount, state, district from free text
ComplaintTextProcessorService	Language detection (Hindi vs English) + Hindi-to-English translation with transliteration + calls parser
ComplaintStoreService	LocalStorage-based reactive store (<code>BehaviorSubject</code>) for filed complaints

OcrService	Mock OCR service (returns sample Hindi/English complaint letters for demo)
------------	--

5.4 Dynamic Form System

The complaint form is entirely **schema-driven**:

1. Backend stores JSON schema in `form_config` table
2. Frontend fetches schema via `GET /api/form-config/raise-complaint`
3. `FileComplaintComponent` iterates `schema.steps[].fields[]`
4. Each field rendered by `DynamicFieldComponent` based on `field.type`
5. Fields support: `optionsSource` (dynamic options from API), `dependsOn` (cascading selects), `conditionalFields` (show/hide)

To add a new field: Update the JSON schema in DB → no frontend code change needed.

5.5 PWA Features

- **Service Worker** (`sw.js`): Network First for API, Cache First for static assets
- **Offline Page** (`offline.html`): User-friendly with retry button
- **Background Sync**: Queues complaint submissions when offline, syncs when online
- **Manifest**: Installable PWA with shortcuts to File/Track complaint

6. Backend Architecture

6.1 Layered Architecture

```

Controller → Service → Repository → Database
    |               |
    |               └─ @Cacheable/@CacheEvict (Redis)
    |
    └─ @Validated (Bean Validation)

```

6.2 Configuration Profiles

Profile	Database	Cache	Tomcat Threads	Use Case
dev	H2 (in-memory)	Simple (ConcurrentMap)	400	Local development
default	MySQL	Redis	400	Integration testing
prod	MySQL (tuned)	Redis (Lettuce pool)	800	Production

To run locally: `mvn spring-boot:run -Dspring-boot.run.profiles=dev`

6.3 Caching Strategy

Cache Name	TTL	Evicted On	Purpose
dashboard	2 min	New complaint filed / status change	Aggregated stats
categories	1 hour	Manual	Complaint category tree
categories-root	1 hour	Manual	Root-level categories

categories-sub	1 hour	Manual	Sub-categories by parent
banks	1 hour	Manual	Bank/entity list
banks-by-type	1 hour	Manual	Banks filtered by type
form-config	6 hours	Schema update	Form JSON schema
email-stats	3 min	Email received/replied	Email stats counters

6.4 Rate Limiting (Dual Layer)

Layer 1 - Application (Bucket4j):

- Per IP: 100 requests/second + 3000/minute burst
- Applies to all `/api/*` paths
- X-Forwarded-For aware (works behind proxy)

Layer 2 - Nginx Gateway:

- General API: 50 req/s (burst 100)
- Search API: 20 req/s (burst 50)
- Write API: 10 req/s (burst 20)
- Connection limit: 100 per IP

6.5 Async Processing

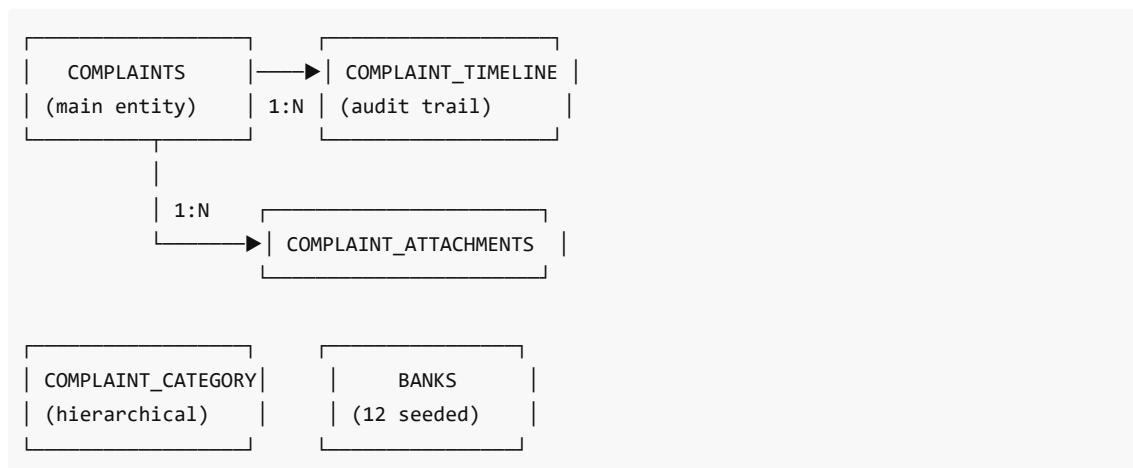
Timeline entries are written asynchronously using `@Async("taskExecutor")` :

- Core pool: 10 threads
- Max pool: 50 threads
- Queue capacity: 500
- Graceful shutdown with 30s timeout

Note for developers: `@Async` won't work when called from within the same class (Spring proxy limitation). If you need to call `addTimelineAsync` from `ComplaintService`, extract it to a separate `TimelineService` class.

7. Database Design

7.1 Entity Relationship



SIMULATED_EMAILS
(email demo)

FORM_CONFIG
(JSON schema)

7.2 Key Indexes (Complaint Table)

Index Name	Columns	Purpose
idx_complaint_number	complaintNumber (unique)	Track lookup
idx_complaint_status	status	Filter by status
idx_complaint_priority	priority	Filter by priority
idx_complaint_email	complainantEmail	User lookup
idx_complaint_category	CATEGORY_ID	Category filter
idx_complaint_bank	BANK_ID	Bank filter
idx_complaint_created	createdAt	Sort by date
idx_complaint_status_created	status, createdAt	Composite: status + date sort

7.3 Complaint Fields

Field	Type	Description
id	Long (auto)	Primary key
complaintNumber	VARCHAR(50)	Unique: CMS-YYYYMMDD-XXXXXX
complainantName	VARCHAR(200)	Filer's name
complainantEmail	VARCHAR(200)	Filer's email
complainantPhone	VARCHAR(20)	Filer's phone
complainantAddress	VARCHAR(500)	Full address
bankId	Long (FK)	Target bank
bankBranch	VARCHAR(300)	Branch name
accountNumber	VARCHAR(100)	Account number
categoryId	Long (FK)	Complaint category
subject	VARCHAR(500)	Brief subject
description	TEXT	Full complaint text
reliefSought	TEXT	What user wants resolved
status	VARCHAR(30)	pending/in_progress/resolved/closed/escalated/rejected

priority	VARCHAR(20)	high/medium/low
filingType	VARCHAR(50)	online/physical/email
bankComplaintReference	VARCHAR(200)	Prior bank complaint ref
bankComplaintDate	DateTime	When filed with bank
assignedOfficer	VARCHAR(200)	Assigned officer name
filedAt, resolvedAt, closedAt, escalatedAt	DateTime	Lifecycle timestamps
createdAt, updatedAt	DateTime	Audit timestamps

8. Scalability & Performance

8.1 Capacity Estimates

Component	Config	Capacity
Nginx	4096 worker connections	~8000 concurrent connections
Backend (x2)	800 threads each	~1600 concurrent requests
MySQL	500 max connections, 1GB buffer	~10K queries/sec
Redis	512MB LRU	~100K ops/sec
Total System		~10K-50K concurrent users

8.2 Performance Optimizations

1. **Read path:** Redis cache → avoids DB hits for categories, banks, dashboard, form config
2. **Write path:** Async timeline writes → main thread returns immediately after complaint save
3. **DB:** Batch inserts enabled (`batch_size: 30-50`), prepared statement caching
4. **HTTP:** Gzip compression (>1KB), keepalive connections, response caching at Nginx
5. **Frontend:** Lazy-loaded routes, tree-shaking, 2MB budget, 1yr static asset cache
6. **Connection reuse:** HikariCP pool (50-100), Nginx keepalive (32 upstream)

8.3 Scaling Playbook

Users	Action
0 - 10K concurrent	Current docker-compose setup (2 backends)
10K - 50K concurrent	Add more backend instances in docker-compose, increase MySQL connections
50K - 200K concurrent	Move to Kubernetes, add MySQL read replicas, Redis cluster
200K+ concurrent	CDN for frontend, DB sharding, dedicated search (Elasticsearch)

9. Infrastructure & Deployment

9.1 Docker Services

Service	Image	Resources	Health Check
cms-mysql	mysql:8.0	-	mysqladmin ping
cms-redis	redis:7-alpine	512MB max	redis-cli ping
cms-backend-1	Custom (JRE 17)	1.5GB / 2 CPU	/actuator/health
cms-backend-2	Custom (JRE 17)	1.5GB / 2 CPU	/actuator/health
cms-frontend	Custom (nginx)	256MB / 0.5 CPU	curl localhost
cms-gateway	nginx:1.27-alpine	256MB / 1 CPU	-

9.2 Deployment Commands

```
# Development (local)
cd cms-frontend && ng serve --port 4200
cd cms-backend && mvn spring-boot:run -Dspring-boot.run.profiles=dev

# Production (Docker)
cd cms-infra
cp .env.example .env # Set passwords
docker-compose up -d --build

# Scale backend
docker-compose up -d --scale cms-backend-1=3
```

9.3 Environment Variables

Variable	Default	Description
DB_URL	jdbc:mysql://localhost:3306/cms_db	MySQL JDBC URL
DB_USERNAME	root	DB user
DB_PASSWORD	root	DB password
REDIS_HOST	localhost	Redis host
REDIS_PORT	6379	Redis port
CMS_ATTACHMENTS_PATH	C:/cms-attachments	File upload path
RATE_LIMIT_RPS	100	Requests per second limit
RATE_LIMIT_CPM	10	Complaints per minute limit

10. API Reference

Complaints

Method	Endpoint	Description	Cache
GET	/api/complaints	List all (optional: ?status=, ?search=)	No
GET	/api/complaints/paged?page=0&size=20	Paginated list	No
GET	/api/complaints/{id}	Get by ID	No
GET	/api/complaints/track/{complaintNumber}	Track by number	No
POST	/api/complaints	File new complaint	Evicts: dashboard
PUT	/api/complaints/{id}	Update status/priority/officer	Evicts: dashboard
DELETE	/api/complaints/{id}	Delete complaint	No
GET	/api/complaints/{id}/timeline	Get audit timeline	No

Categories & Banks

Method	Endpoint	Description	Cache
GET	/api/categories	All active categories	categories (1h)
GET	/api/categories/root	Root categories only	categories-root (1h)
GET	/api/categories/{parentId}/sub	Sub-categories	categories-sub (1h)
GET	/api/banks	All active banks	banks (1h)
GET	/api/banks?type=public	Banks by type	banks-by-type (1h)

Form Config

Method	Endpoint	Description	Cache
GET	/api/form-config/{formKey}	Get form schema JSON	form-config (6h)

Dashboard

Method	Endpoint	Description	Cache
GET	/api/dashboard	Aggregated complaint stats	dashboard (2m)

Email Simulation

Method	Endpoint	Description
POST	/api/email-simulation/receive	Simulate incoming email
POST	/api/email-simulation/reply-with-form	Reply with form link

GET	/api/email-simulation/threads	List email threads
GET	/api/email-simulation/threads/{threadId}	Get thread detail
GET	/api/email-simulation/stats	Email stats

Monitoring

Method	Endpoint	Description	Access
GET	/actuator/health	Health check	Internal only
GET	/actuator/metrics	Metrics	Internal only
GET	/actuator/prometheus	Prometheus scrape	Internal only

11. Developer Guide - How to Modify

11.1 Adding a New Form Field

No code change required. Update the form schema in the database:

1. Connect to DB (or modify `DataInitializer.java` for dev)
2. Update `form_config` table where `form_key = 'raise-complaint'`
3. Add field to the appropriate step in the JSON:

```
{
  "key": "newFieldKey",
  "label": "New Field Label",
  "type": "text|email|tel|select|textarea|radio|file|date",
  "required": true,
  "placeholder": "Enter value"
}
```

4. If `type: "select"` with dynamic options, set `"optionsSource": "yourDataSource"` and handle it in `FileComplaintComponent.getOptionsForField()`
5. Clear Redis cache: `DEL form-config::raise-complaint`

11.2 Adding a New Page/Route

1. Generate component:

```
cd cms-frontend
ng generate component components/my-new-page --standalone
```

2. Add route in `src/app/app.routes.ts` :

```
{ path: 'my-new-page', loadComponent: () => import('./components/my-new-page/my-new-page.component').then(m => m.MyNewPageComponent) }
```

3. Add navigation link in `header.component.html` :

```
<a routerLink="/my-new-page">My New Page</a>
```

11.3 Adding a New API Endpoint

1. Create DTO in `dto/` package if needed
2. Add method in the relevant Service class
3. Add endpoint in the relevant Controller:

```
@GetMapping("/my-endpoint")
public ResponseEntity<MyDto> myEndpoint() {
    return ResponseEntity.ok(myService.doSomething());
}
```

4. If cacheable, add `@Cacheable("cache-name")` and configure TTL in `RedisCacheConfig.java`
5. Add corresponding method in frontend `cms.service.ts`

11.4 Adding a New Database Entity

1. Create entity in `entity/` package with `@Entity` , `@Table` , indexes
2. Create repository interface extending `JpaRepository`
3. Create service with `@Transactional` annotations
4. Create controller with endpoints
5. Run with `ddl-auto: update` (dev) to auto-create table
6. For production: create migration SQL manually

11.5 Modifying Rate Limits

- **Application level:** Change `cms.rate-limit.api-requests-per-second` in `application.yml`
- **Nginx level:** Modify `limit_req_zone` directives in `cms-infra/nginx/nginx.conf`
- **Per-endpoint:** Adjust `burst` values in Nginx location blocks

11.6 Adding a New Cache

1. Add cache name in `RedisCacheConfig.java` :

```
cacheConfigs.put("my-cache", defaultConfig.entryTtl(Duration.ofMinutes(10)));
```

2. Use in service:

```
@Cacheable(value = "my-cache", key = "#param")
public MyData getData(String param) { ... }

@CacheEvict(value = "my-cache", allEntries = true)
public void updateData() { ... }
```

11.7 Changing NLP Parsing (Complaint Text Extraction)

Edit `complaint-parser.service.ts` :

- **Add a new bank:** Add to the dictionary array:

```
{ keywords: ['new bank', 'bank name'], value: 'new_bank', field: 'bankName' }
```

- **Add a new category:** Add keyword mapping + update `categoryMap`
- **Add a new state/district:** Add to dictionary entries with field `'state'` or `'district'`
- **Add Hindi support:** Edit `complaint-text-processor.service.ts` → add to `cityMap`, `bankMap`, `nameMap`

11.8 Mobile Responsiveness

Breakpoints used across the project:

- `1024px` — Tablet landscape (grid collapse, flex direction change)
- `768px` — Tablet portrait / mobile (hamburger menu, stacked layouts)
- `480px` — Small phones (further text/spacing reduction)

Key files to modify for responsive fixes:

- `header.component.scss` — Navigation & top strip
- `home.component.scss` — All landing page sections
- `file-complaint.component.scss` — Form wizard & modal

11.9 Running Tests

```
# Frontend
cd cms-frontend && ng test

# Backend
cd cms-backend && mvn test
```

11.10 Common Development Tasks

Task	What to modify
Change complaint statuses	<code>Complaint.java</code> (enum values), frontend status badges in <code>home.component.scss</code>
Add new bank	<code>DataInitializer.seedBanks()</code> or directly in DB
Add new category	<code>DataInitializer.seedCategories()</code> or directly in DB
Change form validation	Update JSON schema required field + add backend validation in <code>FileComplaintRequest.java</code>
Add file type support	Update accept in form schema JSON + nginx mime types
Change landing page content	<code>home.component.html</code> + <code>home.component.ts</code> (data arrays)
Add new language	Create new translation method in <code>ComplaintTextProcessorService</code>

Scale to more instances

Add `cms-backend-3`, `cms-backend-4` etc. in `docker-compose.yml` + update Nginx upstream

Known Limitations & Future Improvements

1. **@Async self-invocation:** `addTimelineAsync()` called from within `ComplaintService` won't be intercepted by Spring proxy. Extract to separate `TimelineService` for true async behavior.
2. **OCR is mocked:** `OcrService` returns sample text. Integrate with Tesseract or Google Vision API for real OCR.
3. **No authentication:** Currently no login/auth system. Add Spring Security + JWT for production.
4. **No real email integration:** Email simulation is demo-only. Integrate with SMTP/IMAP for production.
5. **PWA icons not generated:** Need to run `npm run pwa-asset-generator` to create actual icon PNGs.
6. **Single MySQL instance:** For high availability, add read replicas and failover.

Quick Reference Commands

```
# Start frontend (dev)
cd cms-frontend && ng serve --port 4200

# Start backend (dev, H2 database)
cd cms-backend && mvn spring-boot:run -Dspring-boot.run.profiles=dev

# Start backend (with MySQL + Redis)
cd cms-backend && mvn spring-boot:run

# Full stack via Docker
cd cms-infra && docker-compose up -d --build

# View logs
docker-compose -f cms-infra/docker-compose.yml logs -f cms-backend-1

# Access H2 Console (dev profile)
http://localhost:8082/h2-console (JDBC URL: jdbc:h2:mem:cms_db)

# Access Actuator
http://localhost:8082/actuator/health
```